

Encryption Method

1. Write a short note explaining the difference between symmetric and asymmetric encryption, and give one real-world example of where each might be used in a cloud app like WhatsApp or Paytm.

ANS

Feature	Symmetric Encryption	Asymmetric Encryption
Key Used	Uses one shared key for both encryption and decryption.	Uses two keys: a public key for encryption and a private key for decryption.
Speed	Faster and suitable for encrypting large amounts of data.	Slower because of complex mathematical operations.
Security	The shared key must be exchanged securely.	No need to share the private key, making key distribution more secure.
Example	AES-256	RSA, ECC

2. Compare server-side encryption and client-side encryption by listing 3 pros and 3 cons for each method in the context of storing user photos on a cloud platform like Instagram.

ANS

Server-Side Encryption (SSE)	Client-Side Encryption (CSE)
Pros	
Easy to configure and manage.	Data is encrypted before leaving the user's device.
Cloud provider manages the encryption keys.	The cloud provider cannot read the encrypted data.
Minimal changes are required in the application.	Provides greater privacy and user control.
Cons	
The cloud provider manages the keys, so users must trust the provider.	The user is responsible for managing encryption keys.
Data is encrypted only after it reaches the cloud.	Losing the encryption key can make data permanently inaccessible.
Limited control over key management (unless using customer-managed keys).	More complex to implement and maintain.

3. Set up an AWS S3 bucket (or use any cloud storage emulator like MinIO if you don't have an AWS account) and enable server-side encryption using AES-256 for all uploaded files. Upload a sample file and verify encryption is active.

ANS

1. Log in to the **AWS Management Console**.
 2. Open **Amazon S3**.
 3. Create or open an existing bucket.
 4. Go to the **Properties** tab.
 5. Under **Default Encryption**, click **Edit**.
 6. Enable **Server-Side Encryption**.
 7. Select:
SSE-S3 (AES-256)
 8. Save the configuration.
 9. Upload a sample file (for example, sample.txt).
 10. Open the uploaded object and verify that **Server-side encryption: AES-256** is displayed.
- **Result:** All newly uploaded objects are automatically encrypted using AES-256.
4. Simulate uploading a playlist cover image to your 'Spotify-style' S3 bucket with server-side encryption enabled, then use the AWS CLI or MinIO client to confirm the encryption status of the uploaded object.

Hint: Use the 'aws s3api head-object' command or MinIO's object info command to check encryption metadata.

ANS

- Upload a sample image:
 - UPLOAD AN CODE
- **Observation**
 - The value:
 - ServerSideEncryption: AES256
- confirms that the uploaded object is encrypted using **AES-256 Server-Side Encryption**.

5. Use ChatGPT or Copilot to generate a code snippet in Python or Node.js that uploads a file to an S3 bucket with server-side encryption enabled. Test the code with your own file and explain how the encryption parameter works.

ANS

- TYPE A PYTHON CODE

❖ Explanation

- **0boto3.client('s3')** creates an AWS S3 client.
- **upload_file()** uploads the local file to the S3 bucket.
- **ExtraArgs** passes additional options to the upload request.
- **ServerSideEncryption='AES256'** tells Amazon S3 to automatically encrypt the uploaded file using the AES-256 algorithm.
- The encryption is handled by AWS, so the application does not need to manage encryption keys.